1. What is Java?

Java is a high-level, object-oriented, general-purpose programming language developed by **Sun Microsystems**.

Java was originally developed by **James Gosling** and released in **1995**.

Why is Java popular?

Java is used for:

- Web applications
- Enterprise applications
- Backend development
- Android development
- Desktop applications
- Banking applications
- Cloud applications
- Big-data technologies
- APIs and microservices

Famous Java principle

> **Write Once, Run Anywhere (WORA)**

This means Java programs can run on different operating systems as long as the required Java Runtime Environment is available.

For example:

```
Java Program
     ↓
Java Compiler
     ↓
Bytecode (.class)
     ↓
JVM
     ↓
Windows / macOS / Linux
```

---

2. Java vs C/C++

Before starting Java, students should understand one important point.

Feature	C	C++	Java
Programming style	Procedural	Object-oriented + procedural	Mainly object-oriented
Platform independent	✗	✗	✓

Feature	C	C++	Java
Pointer support	✓	✓	No direct pointers
Garbage collection	✗	✗	✓
Compilation	Machine code	Machine code	Bytecode
JVM	✗	✗	✓
Security	Lower	Lower	Higher
Exception handling	Limited	✓	✓

---

3. JDK, JRE and JVM

This is one of the **most important concepts** for beginners.

Students often confuse:

- JDK
- JRE
- JVM

Let's understand them separately.

3.1 JVM – Java Virtual Machine

JVM = Java Virtual Machine

JVM is responsible for **running Java bytecode**.

Suppose we write:

`Hello.java`

After compilation:

`Hello.class`

The `.class` file contains **bytecode**.

JVM executes this bytecode.

```
Hello.java
    ↓
javac
    ↓
Hello.class
    ↓
JVM
    ↓
Program Output
```

Important

JVM is **platform dependent**.

The JVM implementation is different for different operating systems.

For example:

```
Windows → Windows JVM
macOS   → macOS JVM
Linux   → Linux JVM
```

But the Java bytecode remains the same.

---

4. JRE – Java Runtime Environment

JRE = Java Runtime Environment

JRE provides the environment required to **run Java applications**.

Conceptually:

```
JRE
 └── JVM
     └── Java Libraries
```

JRE is mainly concerned with **running** Java applications.

---

5. JDK – Java Development Kit

JDK = Java Development Kit

JDK is required when we want to **develop Java applications**.

It contains development tools such as:

- Java compiler
- JVM
- Java libraries
- Debugging/development tools

Conceptually:

```
JDK
├── Development Tools
├── Java Libraries
└── JVM
```

Most important relationship

```
JDK
 └── JVM
```

For modern Java distributions, think of **JDK as the complete developer installation**.

Easy way to remember

```
JDK → Develop
JRE → Run
JVM → Execute
```

---

6. What do we need to install?

To write and execute Java programs, we need:

1. JDK

This is the actual Java development kit.

2. VS Code

VS Code is our **code editor/IDE environment**.

3. Java Extension Pack

This provides Java development features inside VS Code.

---

7. Java Installation on Windows

Step 1 – Download JDK

Go to an official JDK provider such as Oracle Java or Eclipse Adoptium.

For a classroom setup, a current **LTS JDK** is a good choice.

For example:

```
JDK 21
```

or another currently supported LTS version.

---

8. Install JDK

After downloading the Windows installer:

1. Open the installer.
2. Click **Next**.
3. Select installation location.
4. Click **Next**.
5. Complete the installation.

For example, Java may be installed somewhere similar to:

```
C:\Program Files\Java\jdk-21
```

The exact folder depends on the JDK version and installer.

---

9. Verify Java Installation

Open:

Command Prompt

Press:

```
Windows + R
```

Type:

```
cmd
```

Press Enter.

Now run:

```
java -version
```

You should get information about the installed Java version.

Then run:

```
javac -version
```

Example:

```
java version "21..."
javac 21...
```

What are these commands?

```
java
```

is used to launch Java applications.

```
javac
```

is the **Java compiler**.

---

10. Why do we need `javac`?

Suppose we create:

```
Hello.java
```

Java source code cannot directly be executed by the traditional compile-and-run process.

First:

```
javac Hello.java
```

The compiler creates:

```
Hello.class
```

Then:

```
java Hello
```

JVM runs the class.

So:

```
.java
 ↓
javac
 ↓
```

```
.class
 ↓
java
 ↓
Output
```

---

11. Install VS Code

Download VS Code from the official Visual Studio Code website.

Install it normally.

After installation, open VS Code.

---

12. Install Java Extension Pack

Open VS Code.

Go to:

```
Extensions
```

Shortcut:

```
Ctrl + Shift + X
```

Search:

```
Extension Pack for Java
```

Install the Microsoft extension pack.

It provides Java development features such as:

- Java language support
- Code completion
- Debugging
- Testing
- Project support
- Maven support

---

13. Create Your First Java Project

Let's create a simple project.

Create a folder:

```
JavaFirstClass
```

Open this folder in VS Code.

Inside it create:

`Hello.java`

Your folder structure:

```
JavaFirstClass
│
└── Hello.java
```

---

14. First Java Program

Write:

```
public class Hello {

    public static void main(String[] args) {

        System.out.println("Hello World");

    }
}
```

This is our **first Java program**.

---

15. Understanding the Program

Let's understand every part.

`public`

`public`

`public` is an **access modifier**.

It means the member/class/method can be accessed from outside according to Java's access rules.

For our first program, remember:

`public`

means accessible.

---

16. `class`

`class`

A class is a **blueprint/template** for creating objects.

Example:

```
class Student {

}
```

Here:

`Student`

is the class name.

---

17. Class Name

Our class is:

```
Hello
```

Therefore:

```
public class Hello
```

means:

> We are creating a public class named `Hello`.

Important rule

When a class is declared:

```
public class Hello
```

the source file should normally be named:

```
Hello.java
```

So:

```
Class name → Hello
File name  → Hello.java
```

---

18. `main()` Method

This is extremely important.

```
public static void main(String[] args)
```

The `main()` method is the **entry point** for a conventional Java application.

When we run the class, Java starts execution from this method.

Think:

```
Java Program
     ↓
main()
     ↓
Statements execute
```

---

19. Breaking Down `main()`

Let's divide it:

```
public static void main(String[] args)
```

`public`

Access modifier.

`static`

The method belongs to the class rather than requiring an object to invoke it.

`void`

The method does not return a value.

`main`

Method name recognized as the application's starting point.

`String[] args`

An array of strings used to receive command-line arguments.

For beginners, you can initially remember the complete structure as:

```
public static void main(String[] args)
```

---

20. `System.out.println()`

Our program contains:

```
System.out.println("Hello World");
```

This prints text to the console.

Let's break it down conceptually:

```
System
  ↓
out
  ↓
println()
```

`println()` means:

> Print the supplied value and move to the next line.

Example:

```
System.out.println("Hello");
System.out.println("Java");
```

Output:

```
Hello
Java
```

---

21. `print()` vs `println()`

`println()`

```
System.out.println("Hello");
System.out.println("Java");
```

Output:

```
Hello
```

```
Java
```

`print()`

```
System.out.print("Hello");
System.out.print("Java");
```

Output:

```
HelloJava
```

Difference

```
print()   → prints without moving to a new line
println() → prints and moves to a new line
```

---

22. Semicolon `;`

Java statements generally end with:

```
;
```

Example:

```
System.out.println("Hello");
```

The semicolon indicates the end of the statement.

Incorrect:

```
System.out.println("Hello")
```

Correct:

```
System.out.println("Hello");
```

---

23. Curly Braces `{ }`

Java uses curly braces to define blocks.

Example:

```
public class Hello {

}
```

The class body is inside:

```
{
}
```

Similarly:

```
public static void main(String[] args) {

}
```

The method body is inside curly braces.

---

24. Comments in Java

Comments are ignored by the compiler.

Single-line comment

```
// This is a comment
```

Example:

```
// Printing Hello World
System.out.println("Hello World");
```

Multi-line comment

```
/*
   This is a
   multi-line comment
*/
```

---

25. Running Java Program in VS Code

There are two common ways.

Method 1 – Run button

When you open:

```
Hello.java
```

VS Code's Java extension normally provides a **Run** option above the `main()` method.

Click:

```
Run
```

The program executes.

Output appears in the terminal.

---

26. Method 2 – Using Terminal

This method is **very important for students**, because it helps them understand what actually happens.

Open VS Code terminal:

```
Terminal → New Terminal
```

Or use the terminal shortcut available in your VS Code setup.

Navigate to the folder containing:

```
Hello.java
```

Compile:

```
javac Hello.java
```

If compilation succeeds, you'll see a generated:

```
Hello.class
```

Then execute:

```
java Hello
```

Output:

```
Hello World
```

---

27. Very Important: Don't Write `.class`

When running the class, write:

```
java Hello
```

Not:

```
java Hello.class
```

Why?

Because:

```
javac Hello.java
```

uses the source filename.

But:

```
java Hello
```

uses the **class name**.

Remember:

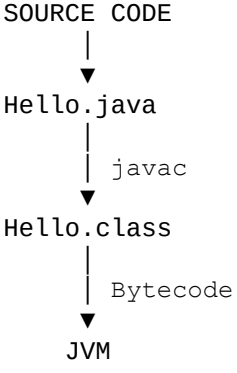
```
Compile:
javac Hello.java

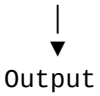
Run:
java Hello
```

---

28. Complete Flow of Java Program

This is one of the most important diagrams for the first class.





Explain to students:

> Java source code is compiled into bytecode. The JVM executes that bytecode.

---

29. What is Bytecode?

The output of the Java compiler is called **bytecode**.

Example:

`Hello.java`

becomes:

`Hello.class`

The `.class` file contains Java bytecode.

Bytecode is designed to be executed by the JVM.

This is a major reason Java supports portability across operating systems.

---

30. Why is Java Platform Independent?

Suppose we write:

```
System.out.println("Hello");
```

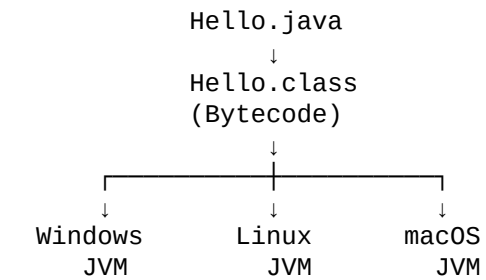
We compile it into:

`Hello.class`

That bytecode can be used with a suitable JVM on:

```
Windows
Linux
macOS
```

Conceptually:



Therefore:

> **Java is platform independent at the bytecode/application level, while the JVM itself is platform-specific.**

---

31. Java Naming Conventions

Students should start following proper naming conventions from day one.

Class

Use PascalCase:

```
Student
Employee
BankAccount
HelloWorld
```

Avoid:

```
student
studentclass
hello_world
```

Variables

Use camelCase:

```
studentName
studentAge
totalMarks
```

Constants

Generally use uppercase with underscores:

```
MAX_VALUE
PI
MINIMUM_AGE
```

---

32. First Program with Multiple Statements

Try:

```
public class Hello {

    public static void main(String[] args) {

        System.out.println("Hello World");
        System.out.println("Welcome to Java");
        System.out.println("This is my first Java class");

    }
}
```

Output:

```
Hello World
Welcome to Java
This is my first Java class
```

---

33. Printing Numbers

We can also print numbers.

```
public class Demo {

    public static void main(String[] args) {

        System.out.println(10);
        System.out.println(20);
        System.out.println(100);

    }
}
```

Output:

```
10
20
100
```

Notice:

```
"10"
```

is text/string, whereas:

```
10
```

is a number.

---

34. Printing Calculations

Java can perform calculations.

```
public class Calculator {

    public static void main(String[] args) {

        System.out.println(10 + 20);
        System.out.println(50 - 20);
        System.out.println(10 * 5);
        System.out.println(20 / 5);

    }
}
```

Output:

```
30
30
50
4
```

---

35. Printing Text + Number

We can combine them.

```
public class Student {

    public static void main(String[] args) {

        System.out.println("Age: " + 22);

    }
}
```

Output:

```
Age: 22
```

Here:

```
+
```

can perform addition or string concatenation depending on the operands.

---

36. Example: Student Information

```
public class Student {

    public static void main(String[] args) {

        System.out.println("Student Name: Akshay");
        System.out.println("Age: 22");
        System.out.println("Course: Java Full Stack");
        System.out.println("College: ABC College");

    }
}
```

This is a good first practical exercise.

---

37. Common Errors Beginners Make

Error 1 – Wrong file name

Class:

```
public class Hello
```

File:

```
Demo.java
```

This causes an error because the public class name and filename don't match.

Correct:

```
Hello.java
```

with:

```
public class Hello
```

Error 2 – Missing semicolon

Wrong:

```
System.out.println("Hello")
```

Correct:

```
System.out.println("Hello");
```

Error 3 – Wrong capitalization

Java is **case-sensitive**.

Wrong:

```
System.Out.Println()
```

Correct:

```
System.out.println()
```

Similarly:

```
String
```

is different from:

```
string
```

---

38. Java is Case Sensitive

These are different:

```
Hello
hello
HELLO
```

Similarly:

```
System
system
```

are different identifiers.

---

39. What Happens When We Click Run?

When we click **Run** in VS Code, conceptually:

```
VS Code
   ↓
Java compiler
   ↓
Compile Java code
   ↓
Bytecode
   ↓
JVM
   ↓
Execute program
   ↓
Terminal
```

VS Code is mainly providing the development environment and tooling. **Java/JDK actually compiles and runs the program.**

---

40. VS Code vs JDK

Students often ask:

> "If I have VS Code, why do I need JDK?"

Because they are different things.

VS Code

```
Editor / Development Environment
```

Used for:

- Writing code
- Editing code
- Debugging
- Extensions
- Project management

JDK

```
Java Development Kit
```

Provides the Java development tools required to compile and run Java programs.

Therefore:

```
VS Code + Java Extension Pack + JDK
                ↓
       Java Development Setup
```

---

41. Important Commands for Students

Command	Purpose
`java -version`	Check Java runtime version
`javac -version`	Check compiler version
`javac Hello.java`	Compile Java source
`java Hello`	Run compiled class
`cd folder`	Change directory
`dir`	List files on Windows
`ls`	List files on macOS/Linux

---

42. Troubleshooting

Problem: `java` is not recognized

Example:

```
'java' is not recognized...
```

Possible reason:

- JDK isn't installed correctly, or
- Java isn't available on the system PATH.

Check the JDK installation and environment configuration.

Problem: `javac` is not recognized

This usually means the Java compiler isn't available in the command-line PATH.

Check:

```
javac -version
```

If it fails, verify the JDK installation and PATH.

---

43. PATH – Basic Understanding

Students don't need to master environment variables in the first class, but they should understand the concept.

When we type:

```
java
```

the operating system needs to know where the Java executable is located.

The **PATH environment variable** contains locations where executable programs can be found.

Conceptually:

```
PATH
 ↓
Java bin folder
 ↓
java
javac
```

---

44. First-Class Practice Questions

Q1. What is Java?

Answer:
Java is a high-level, general-purpose, object-oriented programming language.

Q2. Who developed Java?

Answer:
James Gosling and the team at Sun Microsystems developed Java.

Q3. What is JDK?

Answer:
JDK stands for Java Development Kit and provides tools required to develop Java applications.

Q4. What is JVM?

Answer:
JVM stands for Java Virtual Machine and executes Java bytecode.

Q5. What is JRE?

Answer:
JRE stands for Java Runtime Environment and provides the environment needed to run Java applications.

Q6. What is bytecode?

Answer:
Bytecode is the intermediate code generated by the Java compiler and stored in `.class` files.

Q7. Which command compiles Java code?

`javac`

Q8. Which command runs a Java class?

`java`

Q9. What is the extension of a Java source file?

`.java`

Q10. What is the extension of compiled Java bytecode?

`.class`

---

45. Practical Class Exercise

Ask students to create:

`StudentInfo.java`

with the following output:

```
========================
     STUDENT INFO
========================

Name: Rahul
Age: 21
Course: Java Full Stack
College: ABC College
City: Delhi

========================
```

Students should write the program themselves.

---

46. Homework

Task 1

Create:

```
HelloWorld.java
```

Print:

```
Hello World
Welcome to Java
I am learning Java
```

Task 2

Create:

```
Student.java
```

Print:

```
Name
Age
Course
College
City
```

Task 3

Create:

```
Calculator.java
```

Print the result of:

```
10 + 20
50 - 10
5 * 6
100 / 5
```

Task 4

Create:

```
Employee.java
```

Print:

```
Employee Name
Employee ID
Department
Salary
Company
```

---

47. First-Class Summary

Students should remember these points from Class 1:

```
Java
 ↓
```

```
Programming Language

JDK
 ↓
Used for Java Development

JRE
 ↓
Runtime Environment

JVM
 ↓
Executes Bytecode

.java
 ↓
Java Source Code

javac
 ↓
Compiler

.class
 ↓
Bytecode

java
 ↓
Runs Java Class
```

Most important command sequence

```
javac Hello.java
java Hello
```

Most important Java structure

```
public class Hello {

    public static void main(String[] args) {

        System.out.println("Hello World");

    }
}
```

Execution flow

```
Java Source Code
       ↓
   Compiler
   (javac)
       ↓
   Bytecode
   (.class)
       ↓
     JVM
       ↓
    Output
```